

Topic

Bridging the Chasm: Practical AI Experiments in the Servo Engine



Gregory Terzian

Founder, FOSS(Shanghai) Technology



GOSIM Paris 2026

May 5-6, 2026 Station F, Paris



Bridging the Web API Gap

Practical AI Experiments in the Servo Engine

Gregory Terzian
github.com/gterzian



CONTENTS

GOSIM Paris 2026

Part 01

Implement WebNN

Part 02

Implement CSS Subgrid

Part 03

What comes next ?



01

Implementing WebNN



What is WebNN?

Web Neural Network API

www.w3.org/TR/webnn/

“a web-friendly hardware-agnostic abstraction layer that makes use of Machine Learning capabilities of operating systems and underlying hardware platforms without being tied to platform-specific capabilities”

As of now only supported in Chromium a feature flag.



Structure of a WebNN Implementation

- split between a front-end running in the web content process, and a privileged backend with access to OS ML capabilities.
- Uses Rustnn: github.com/rustnn/rustnn
 - operation types and the graph
 - backend compilation and compute
- Backend CoreML only because so far developed on a Mac only



WebNN LeNet Example

https://webmachinelearning.github.io/webnn-samples/lenet/

WebNN LeNet Example

 Handwritten Digits Classification Samples ▾

Backend

WebNN (CPU)

WebNN (GPU)



Next

Clear

Predict

Prediction Result:

#	Label	Probability
1	3	100.00%
2	9	0.00%
3	8	0.00%

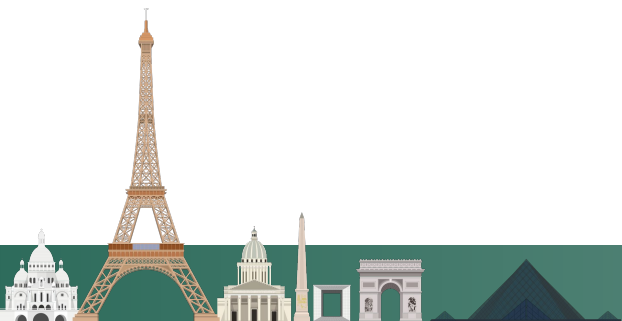
Build Time: 108.79 ms

Execution Time: 437.61 ms



Status of a WebNN Implementation in Servo: blocked

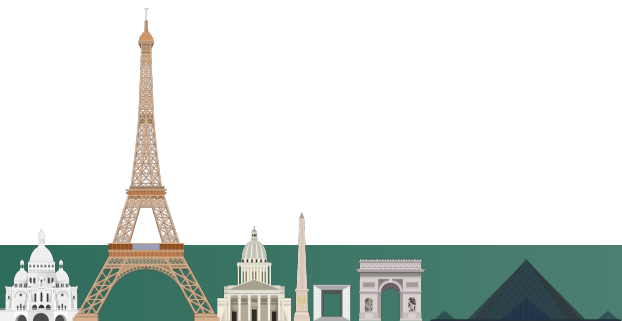
Code written with AI assistance, and so cannot merge into Servo main because of policy disallowing any AI contributions.



Making the Servo codebase more agent ready(1)

What is agent ready? My opinion:

- Put all documentation in the repo itself
- Structure documentation in a way that makes it available when the agent needs it, and not otherwise
- Have the agent add documentation overtime.



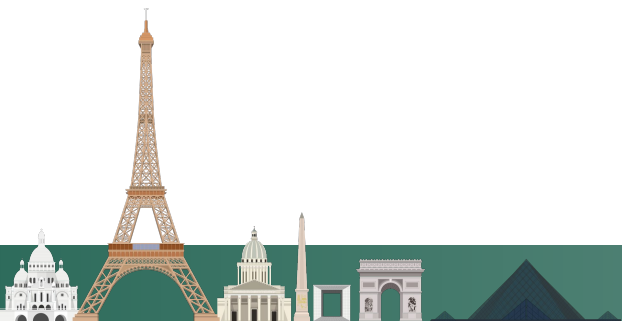
Making the Servo codebase more agent ready(2)

Example: One top-level [AGENTS.md](#), and one [README.md](#) per directory that needs one.

For the Webnn work specifically:

- components/script/[README.md](#): stuff that matters for all script
- components/script/dom/webnn/[README.md](#): stuff that matters only for webnn

At start of each task, agent loads readme files that are relevant only, based on a prompt found in the top-level agents file. At the end of a task, agent writes lessons learned in the most relevant readme.



Status of a WebNN Implementation

Many changes to rustnn library merged: coreml fixes

Commits on Mar 18, 2026

coreml: fix various boolean operations (#72) ...

gterzian and mtavenrath authored on Mar 18 · ✓ 2 / 4

core ml: implement gather elements (#69) ...

gterzian and mtavenrath authored on Mar 18 · ✓ 4 / 6

Commits on Mar 17, 2026

coreml: fix regressions (#68) ...

gterzian and mtavenrath authored on Mar 17 · ✓ 2 / 4

add epsilon to reciprocal (#66) ...

tarekziade and gterzian authored on Mar 6 · ✓ 2 / 4

Gemm handling (#65) ...

tarekziade and gterzian authored on Mar 6 · ✓ 2 / 4

coreml: rename round to roundeven (#61) ...

gterzian and tarekziade authored on Mar 6 · ✓ 4 / 6

Update make test to run fmt and backend report drift check (#64) ...

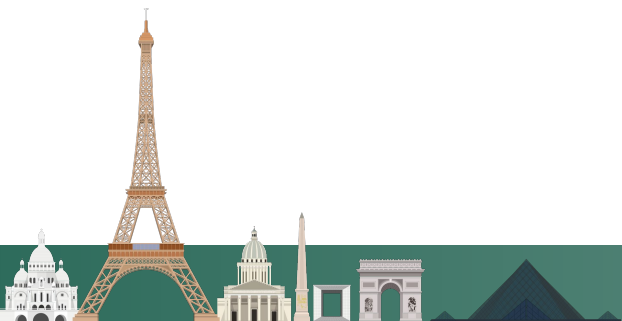
tarekziade authored on Mar 6 · ✓ 2 / 4

coreml: pool ops fixes (#63) ...

gterzian authored on Mar 6 · ✓ 2 / 4

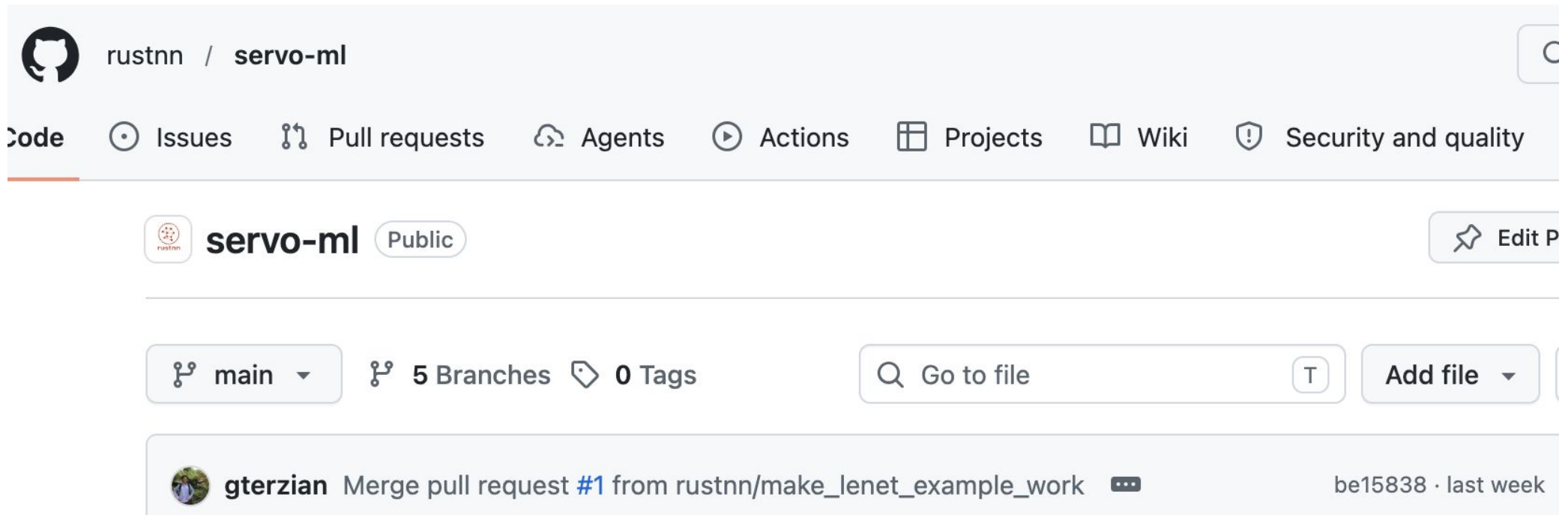
Fix casing for argMax and argMin in coreml converter (#58) ...

gterzian authored on Mar 6 · ✓ 2 / 4



Status of a WebNN Implementation

A fork of Servo containing the work lives now under rustnn:
github.com/rustnn/servo-ml



Status of a WebNN Implementation

rustnn.github.io/servo-ml/

Servo-ML WebNN WPT Report

Generated from wptreport JSON and compared with latest Chrome master+experimental run.
Generated: 2026-04-27 18:02:56 UTC



SERVO SUBTESTS

11,850

SERVO PASSED

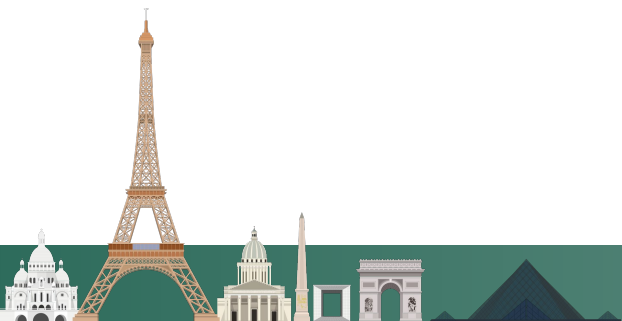
5,593

SERVO PASS RATE

47.2%

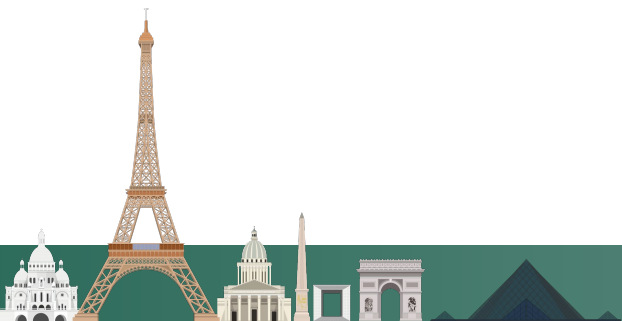
SERVO COVERAGE VS CHROME TOTAL

74.2%



Reflecting on AI use in a WebNN Implementation

- > 10k LOC in one month.
- Personal estimate to reach same status without AI: ~ 6 months.
- Code up to my own standards.
- Took a lot of reviews and iterations to reach where code is today.
- Experience implementing this kind of API in Servo, which enabled AI guidance.



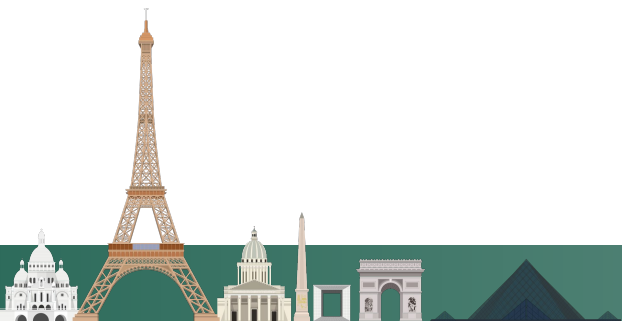
Key Takeaway

AI is a productivity booster, but code review is still essential

Bottlenecks:

- Spec intent (specs are informal; AI reads them literally)
- Architectural decisions not captured in the spec
- Codebase-wide patterns and consistency

Once a bottleneck is resolved, the AI can apply the lesson going forward — shifting the human role from line-by-line reviewer to architectural guide.



02

Implementing CSS Subgrid

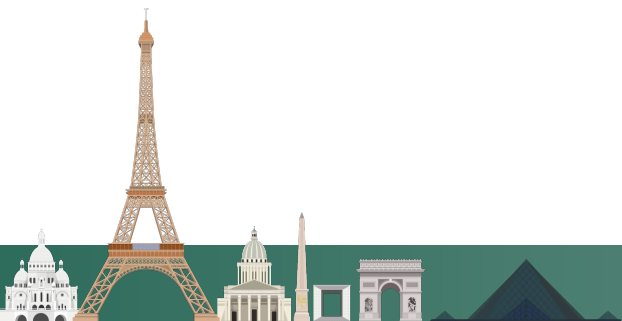


What is CSS Subgrid?

drafts.csswg.org/css-grid/

“Grid Layout is a layout model for CSS that has powerful abilities to control the sizing and positioning of boxes and their contents.”

Baseline Widely Available

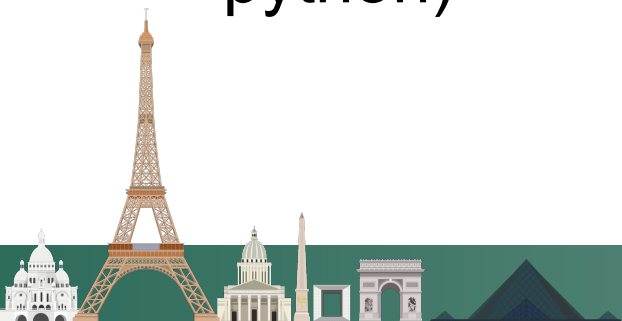


Structure of the spec and implementation?

To be honest: I have no idea => in other words: this was vibe coding.

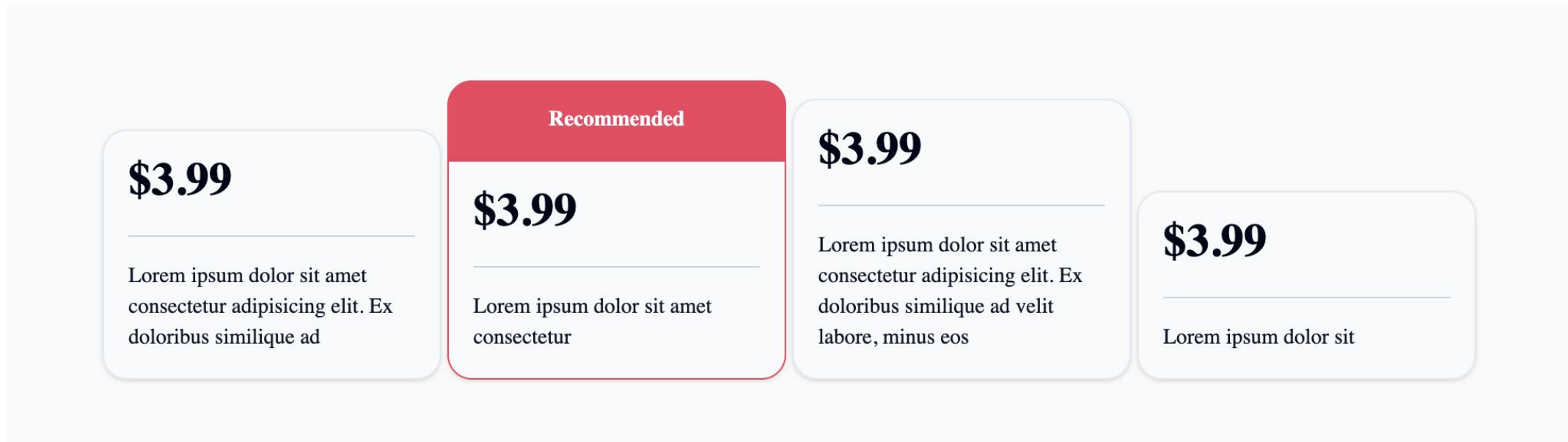
What I did do:

- setup a kind of local harness with:
 - the spec
 - the wpt tests
 - an example from a website I found
 - some additional “agentic” tooling (example: debug css retest from python)

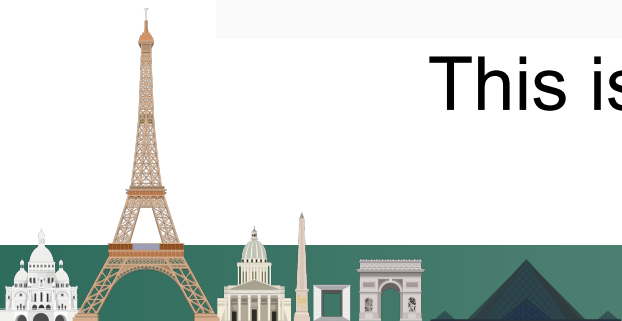


An example of using subgrid:

www.typeonce.dev/article/the-example-that-made-me-get-css-subgrid

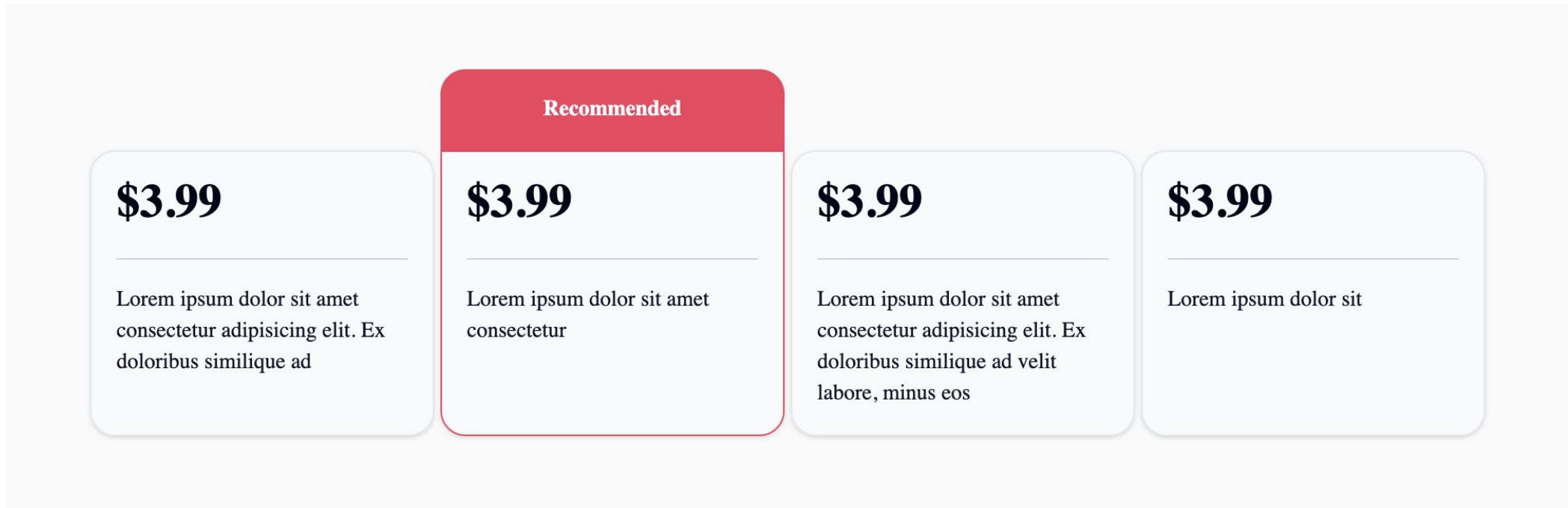


This is grid without subgrid: labels not aligned

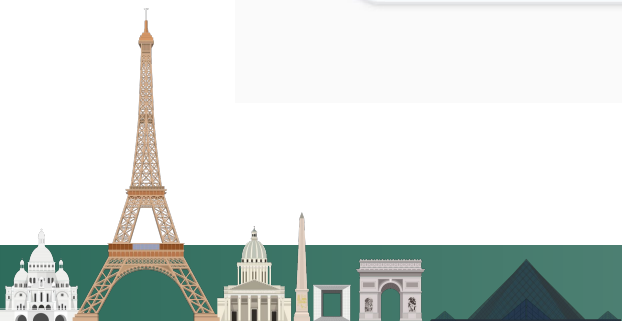


An example of using subgrid:

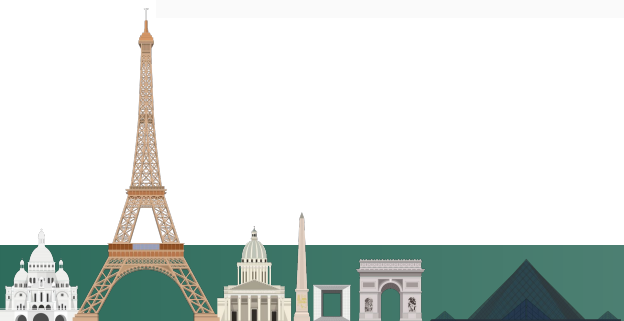
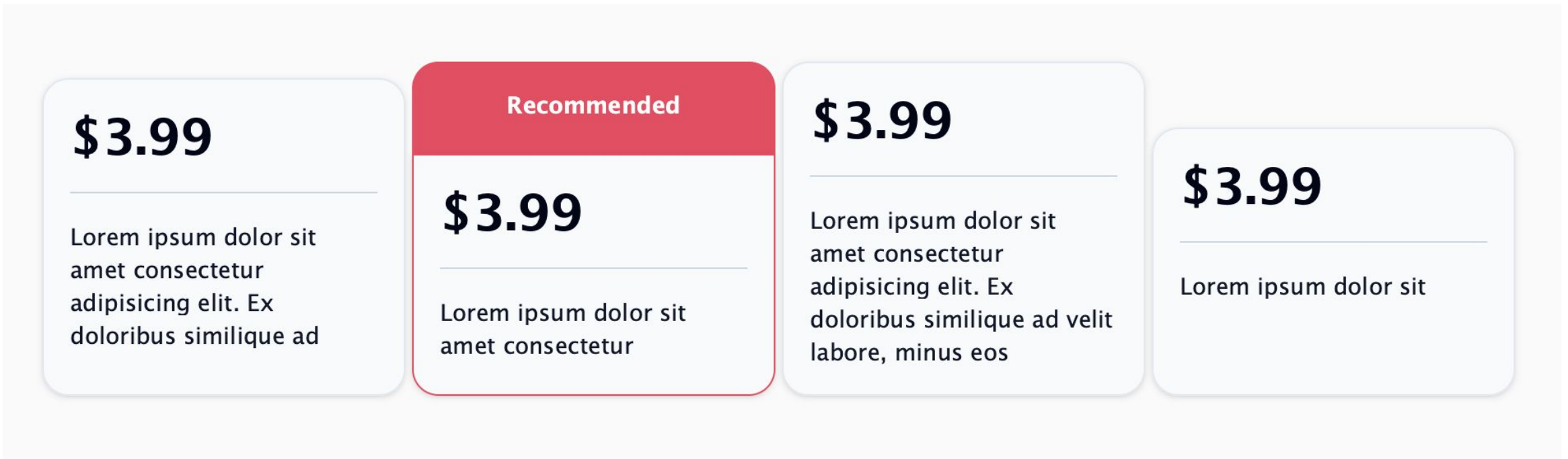
www.typeonce.dev/article/the-example-that-made-me-get-css-subgrid



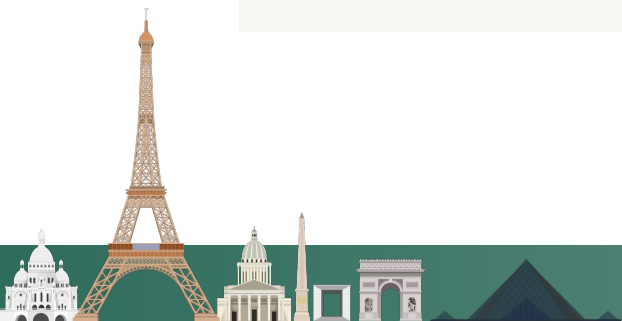
With subgrid: looks good.



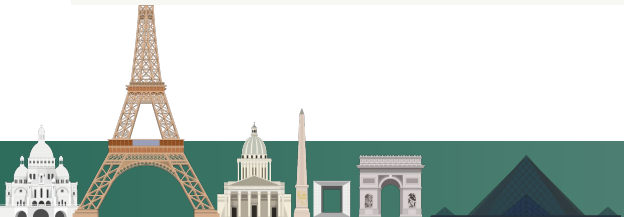
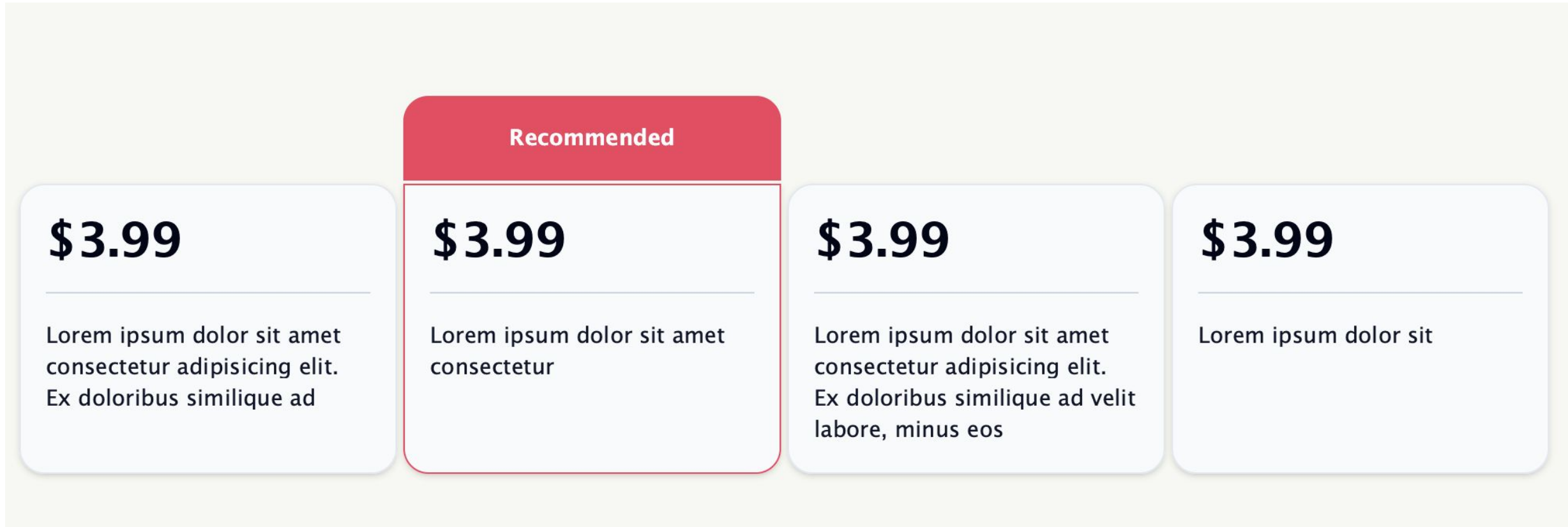
How the example looks in Servo today:



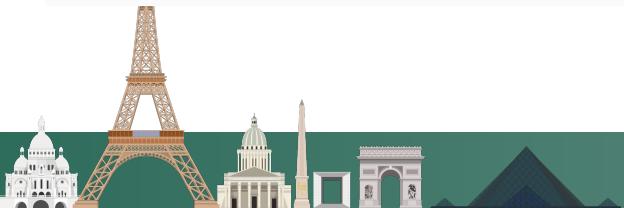
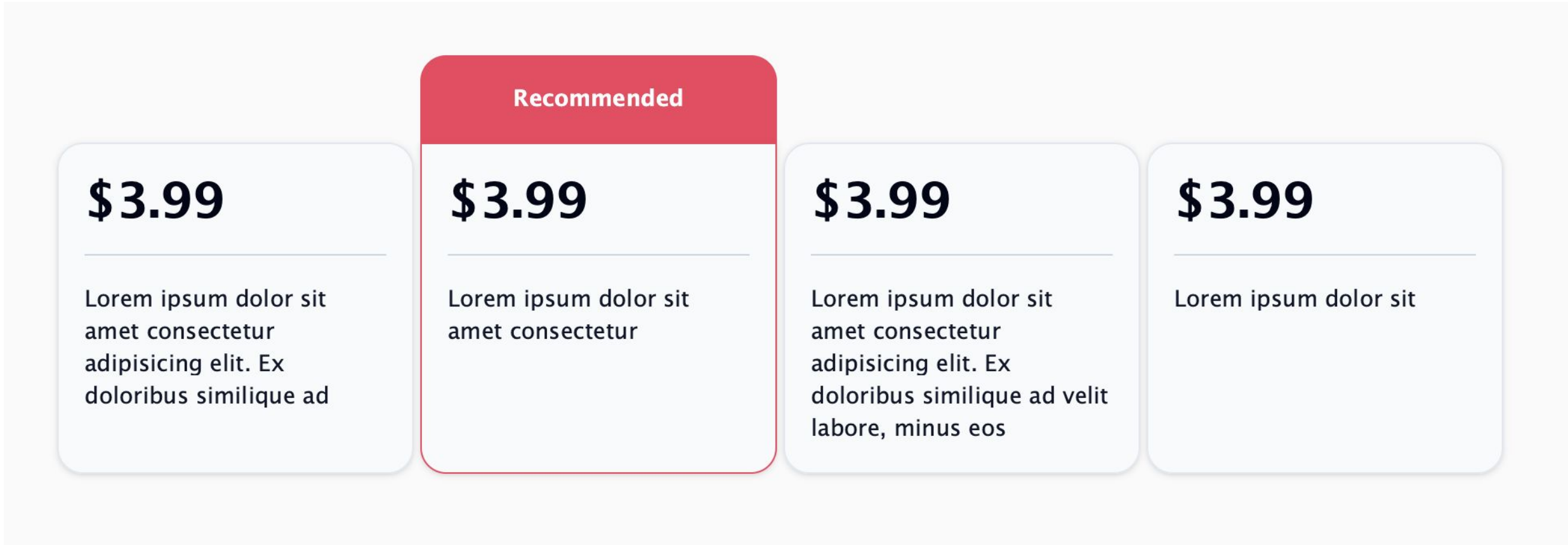
My vibe coded fix: take one



My vibe coded fix: take two



My vibe coded fix: take three

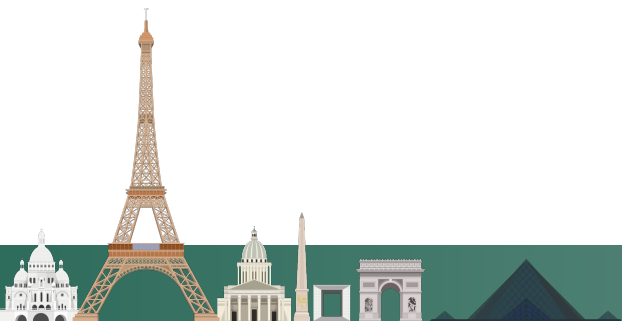


But what about the code?

A nice demo is not the same as a path towards implementing Subgrid as a whole.

Most changes made to Taffy, the layout library.

Initial plan: submit everything one modular piece of work at a time.



First PR: 100 loc. Status: merged after a rewrite from maintainer.



nicoburns commented 3 weeks ago

Member ...

I've minimised your change as `main...nicoburns:taffy:auto-repeat-minimised`

Seems like there are two key improvements:

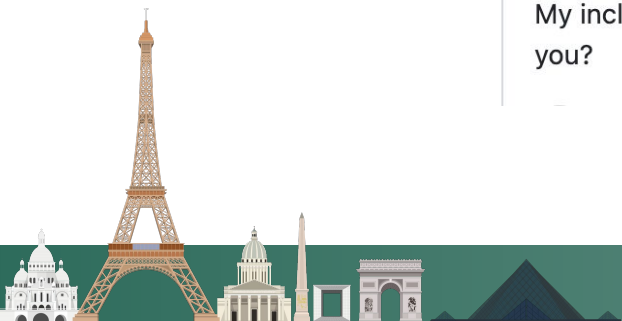
1. Account for regular (non-auto fixed integer) repeats in `non_auto_repeated_track_count`
2. A fix for the of collapsing auto-fit tracks.

I didn't understand what 2 was doing (and the nested loop seemed suspicious), so I dug into this a bit. Turns out the code was in the loop but only ever ran on the last iteration of the loop. So I've pulled it out of the loop, and added a comment to explain what's going on.

The following was not necessary:

- Saturating math (this is not a terrible idea in general, but if we do it I'd rather do it for the whole module at once and also consider things like bumping intermediate variables up to 32-bit types).
- `step_by(2)` - this is potentially a small perf opt, but it doesn't affect output.

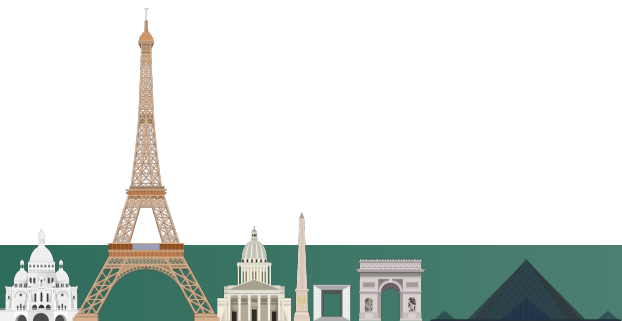
My inclination is to merge that patch rather than this one (with co-authorship credit to you). Does that sounds ok to you?



How much code left to submit in order to match the demo?

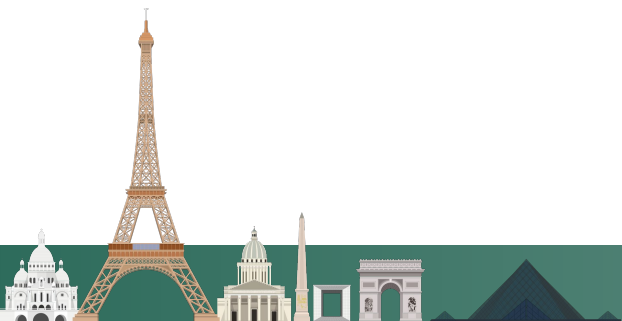
2-3k LOC.

Nico did take look at the code dump as a whole, and said it had some obvious flaws but wasn't that bad. Perhaps it will be incorporated in Taffy in some way...



Key Takeaways from Parts 1 & 2

- Using AI well is real work — it requires domain expertise and careful review
- Vibe coding produces working demos, but not mergeable code



03

What comes next?



Formal methods comeback?

Formal methods might make it to the mainstream and help reduce the review burden from AI generated code.

Why? AI cheapens formal methods, while also making them necessary.

Basic idea: write a formal spec (or review one written by AI), generate code and a proof that the code implements the spec.

In theory, you only have to review the spec.



martin.kleppmann.com/2025/12/08/ai-formal-verification.html

Martin Kleppmann

[Student Projects](#) [About/Contact](#) [Supporters](#)

Prediction: AI will make formal verification go mainstream

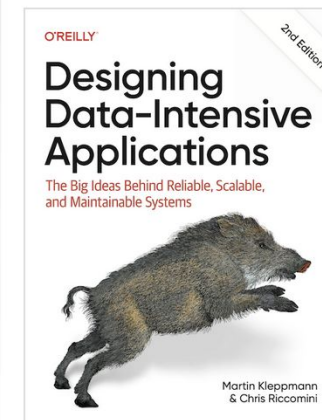
Published by Martin Kleppmann on 08 Dec 2025.

Much has been said about the effects that AI will have on software development, but there is an angle I haven't seen talked about: I believe that AI will bring formal verification, which for decades has been a bit of a fringe pursuit, into the software engineering mainstream.

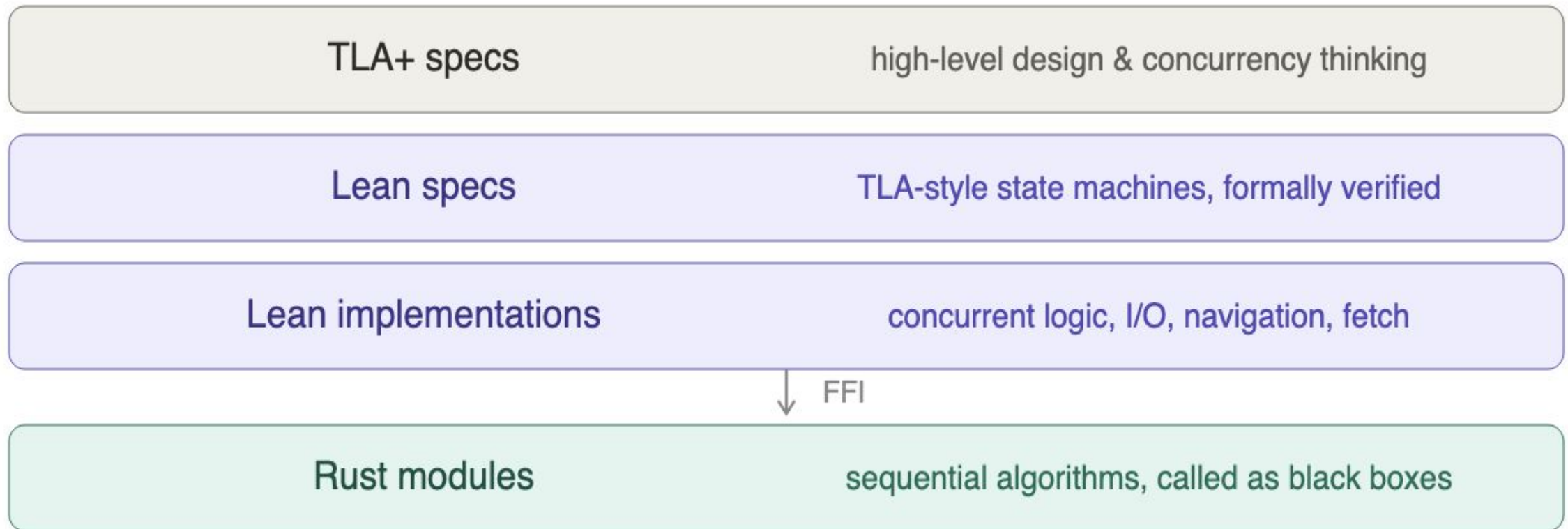
Proof assistants and proof-oriented programming languages such as **Rocq**, **Isabelle**, **Lean**, **F***, and **Agda** have been around for a long time. They make it possible to write a formal specification that some piece of code is supposed to satisfy, and then mathematically prove that the code *always* satisfies that spec (even on weird edge cases that you didn't think of testing). These tools have been used to develop some large formally

SUBSCRIBE

My book



Example: a formally verified Web engine:
github.com/gterzian/formal-web



github.com/gterzian/formal-web

The problem

Some of the hardest bugs in browsers are in the coordination. Navigation races, session history corruption, fetch ordering errors. These are concurrency bugs.

The approach

formal-web moves as much of this logic as possible into a verified Lean kernel. The kernel handles every concurrent decision — navigation, session history, fetch — using formally proven state machines. Rust modules handle the rest: DOM, layout, rendering, JavaScript.



github.com/gterzian/formal-web

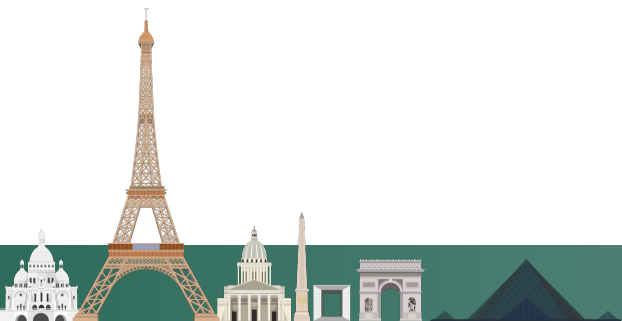
Modularity

The engine composes best-in-class open components:

- **Blitz** — DOM and layout
- **Vello** — GPU-accelerated rendering
- **Wasmtime** — WebAssembly runtime
- **Boa** — JavaScript engine

Security

The process model is designed to meet Apple's [architectural requirements](#) for an independent web engine on iOS — from day one



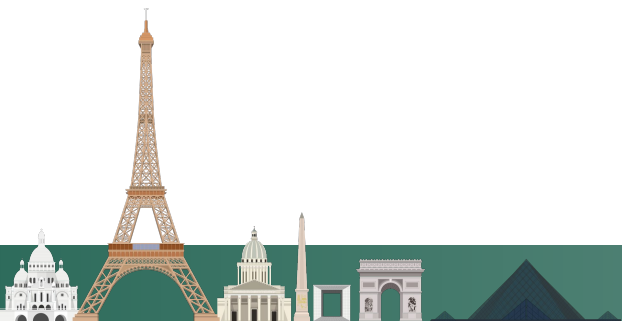
github.com/gterzian/formal-web

The bet

A formally verified core plus composable Rust modules is a tractable path to a correct, secure, and performant web engine.

The hedge

If the Lean turns out to be a bottleneck, it can be translated to Rust (using AI), and then form the basis for verifying the Rust.



Thanks!

